Web Scraper for Digital Ocean

A Flask-based web scraper that extracts data from HTML tables and provides a filtering API. Designed to run on Digital Ocean droplets.

Features

- Scheduled web scraping via cron
- Local file storage for scraped data
- REST API for data filtering
- Automated setup script for Digital Ocean droplets

Prerequisites

- Ubuntu/Debian-based Digital Ocean droplet
- Python 3.8+
- Root access to the droplet

Quick Start

1. Upload the files to your Digital Ocean droplet:

   ```
   scp app.py requirements.txt setup.sh scrape.sh root@your-droplet-ip:~/
   ```

2. SSH into your droplet:

   ```
   ssh root@your-droplet-ip
   ```

3. Make the setup script executable and run it:

   ```
   chmod +x setup.sh
   ./setup.sh
   ```

Configuration

Edit the following variables in `setup.sh` before running:

bash

```bash
APP_NAME="web-scraper"                # Name of your application
APP_DIR="/opt/$APP_NAME"              # Installation directory
USER="$USER"                          # User to run the service as
DATA_DIR="/var/lib/$APP_NAME/data"    # Directory to store scraped data
TARGET_URL="https://example.com/table-page"  # Website to scrape
PORT=5000                             # Port for the web service
CRON_SCHEDULE="0 0 * * *"             # Cron schedule (daily at midnight)
```

Manual Scraping

To run the scraper manually:

```
./scrape.sh
```

Local Development and Testing

1. Create a virtual environment:

   ```
   python3 -m venv venv
   source venv/bin/activate
   ```

2. Install dependencies:

   ```
   pip install -r requirements.txt
   ```

3. Run the scraper:

   ```
   python app.py scrape
   ```

4. Run the web service for development:

   ```
   flask run
   ```

API Endpoints

GET /scrape

Manually triggers the scraping process.

Response:

```json
{
  "success": true,
  "message": "Data scraped and saved",
  "record_count": 42,
  "timestamp": "2023-01-01T12:00:00.123456"
}
```

POST /data

Retrieves and filters the scraped data.

Request Body:

```json
{
  "field1": "value1",
  "field2": "value2"
}
```

Response:

```json
{
  "data": [
    {
      "field1": "value1",
      "field2": "value2",
      "field3": "value3"
    }
  ],
  "count": 1,
  "metadata": {
    "scraped_at": "2023-01-01T12:00:00.123456",
    "record_count": 42
  }
}
```

System Components

1. **Flask Web Service**: Runs continually via systemd, providing the API endpoints
2. **Cron Job**: Runs the scraper on a scheduled basis
3. **Data Storage**: JSON file stored on the filesystem

Customizing the Scraper

Modify the `scrape_and_save_data` function in `app.py` to adjust the scraping logic. The default implementation looks for an HTML table element and extracts rows and columns.

Update the CSS selector to target the specific table on your target website:

```python
# Find the table - modify selector based on actual HTML structure
table = soup.select_one('table.your-table-class')  # Adjust selector as needed
```

Logs and Monitoring

- Web service logs: `journalctl -u web-scraper.service`
- Scraper logs: `/var/log/web-scraper/scrape.log`

Troubleshooting

- **Service won't start**: Check logs with `journalctl -u web-scraper.service`
- **Cron job not running**: Check cron logs with `grep CRON /var/log/syslog`
- **Permission issues**: Ensure proper ownership with `chown -R $USER:$USER $APP_DIR $DATA_DIR`
- **Scraping issues**: Check the scraper logs at `/var/log/web-scraper/scrape.log`

License

MIT